GUÍA DE LABORATORIO: INTEGRACIÓN NUMÉRICA

Física Computacional I — Regla del Trapecio y Simpson en Sistemas Físicos Realistas

Estudiante: _____ **Fecha:** _____

Grupo / Sección: _____ **Calificación:** _____

---

1 OBJETIVOS DE LA PRÁCTICA

- Aplicar las reglas de integración del Trapecio Compuesto y Simpson 1/3 Compuesto a problemas de la mecánica clásica.
- Calcular analíticamente las cotas superiores de error numérico y determinar el tamaño de partición n requerido.
- Programar los algoritmos en Python y verificar experimentalmente los órdenes de convergencia $\mathcal{O}(h^2)$ y $\mathcal{O}(h^4)$.

2 MARCO TEÓRICO Y PROBLEMA FÍSICO

Para un péndulo simple de longitud L, la aproximación de pequeñas oscilaciones da un período $T_0 = 2\pi\sqrt{(L/g)}$. Para amplitudes elevadas θ_0, el período exacto viene dado por la integral elíptica completa de primera especie:

$$T(\theta_0) = 4\sqrt{(L/g)}\int_0^{\pi/2} [\,1 / \sqrt{(1 - k^2 \sin^2\varphi)}\,]\, d\varphi\,, \qquad \text{donde} \quad k = \sin(\theta_0 / 2)$$

3 PARTE 1: ANÁLISIS ANALÍTICO DEL ERROR

Parámetros del sistema: $L = 1.0$ m, $g = 9.81$ m/s^2, amplitud inicial $\theta_0 = 60°$ ($\pi/3$ rad).

Tarea 1.1: Escribe los parámetros de la integral a resolver:
- Parámetro $k = \sin(30°)$ = 0.5
- Valor del período aproximado $T_0 = 2\pi\sqrt{(1.0 / 9.81)} = 2.006066$ segundos.

Sabiendo que para f(φ) = (1 − 0.25 sin 2φ) −1/2 en el intervalo [0, π/2], las cotas superiores de las derivadas son M2 = max |f''(φ)| ≤ 0.75 y M4 = max |f (4) (φ)| ≤ 3.5:

Método Numérico	Cota Teórica del Error	Cálculo Mínimo de n para Error < 10^{-4}
Trapecio Compuesto	$\lvert E_T\rvert \le [\,(b-a)^3 / (12\,n^2)\,]\,M_2$	n mínimo = 50
Simpson 1/3 Compuesto	$\lvert E_S\rvert \le [\,(b-a)^5 / (180\,n^4)\,]\,M_4$	n mínimo (par) = 8

4 PARTE 2: IMPLEMENTACIÓN DE ALGORITMOS EN PYTHON

Completa el código fuente para implementar las funciones de integración numérica utilizando vectorización de NumPy.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import quad

# --- Parámetros Físicos ---
L = 1.0
g = 9.81
theta_0 = np.radians(60)
k = np.sin(theta_0 / 2.0)

def  integrando(phi):
    """Función f(phi) para la integral elíptica del péndulo."""
    return 1.0 / np.sqrt(1.0 - (k**2) * (np.sin(phi)**2))

def trapecio_compuesto(f, a, b, n):
    """Calcula la integral usando la Regla del Trapecio Compuesto."""
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    # COMPLETAR: Aplica la fórmula del Trapecio
    integral = (h / 2.0) * (y[0] + 2 * np.sum(y[1:-1]) + y[-1])
    return integral

def simpson_13_compuesto(f, a, b, n):
    """Calcula la integral usando Simpson 1/3 (n debe ser par)."""
    if n % 2 != 0:
        raise ValueError("El número de subintervalos n debe ser PAR.")
    h = (b - a) / n
    x = np.linspace(a, b, n + 1)
    y = f(x)
    # COMPLETAR: Suma de índices impares y pares
    suma_impares = np.sum(y[1:-1:2])
    suma_pares = np.sum(y[2:-2:2])
    integral = (h / 3.0) * (y[0] + 4 * suma_impares + 2 * suma_pares + y[-1])
    return integral
```

5 PARTE 3: ANÁLISIS DE CONVERGENCIA DEL ERROR

Escribe la rutina para evaluar el error absoluto frente al valor exacto devuelto por `scipy.integrate.quad` para valores de $n = [2, 4, 8, 16, 32, 64, 128, 256]$.

```
# Valor de referencia con alta precisión
I_exacto, _ = quad(integrando, 0, np.pi/2)

n_valores = np.array([2, 4, 8, 16, 32, 64, 128, 256])
err_trap = []
err_simp = []

for n in n_valores:
    I_t = trapecio_compuesto(integrando, 0, np.pi/2, n)
    I_s = simpson_13_compuesto(integrando, 0, np.pi/2, n)
    err_trap.append(abs(I_t - I_exacto))
    err_simp.append(abs(I_s - I_exacto))

# Gráfica Log-Log
plt.figure(figsize=(7,  4))
```

6. RESULTADOS NUMÉRICOS OBTENIDOS

Registra los valores computados en la ejecución con $n = 100$ subintervalos:

Concepto	Valor Calculado	Unidades / Observación
Integral I (Trapecio, n=100)	1.685750354813	Adimensional
Integral I (Simpson 1/3, n=100)	1.685750354813	Adimensional
Período Real $T(60°)$	2.152874666881	Segundos (s)
Error Relativo vs T_0	6.819161	% de subestimación del modelo lineal

7. PREGUNTAS DE DISCUSIÓN Y CONCLUSIONES

Pregunta 1: Análisis de las Pendientes en Log-Log

A partir del gráfico generado, explica cómo se relacionan la pendiente visual de cada curva con el orden teórico del error $O(h^2)$ y $O(h^4)$.

Rta: En una gráfica Log-Log del error frente al número de subintervalos, el método del Trapecio Compuesto presenta teóricamente una pendiente de −2, mientras que Simpson 1/3 Compuesto presenta una pendiente de −4. Esto indica que Simpson tiene un orden de convergencia mayor. Sin embargo, en esta práctica las pendientes pueden diferir debido a las características de la función y a los límites de precisión numérica.

Pregunta 2: Comparación de Eficiencia Computacional

Para alcanzar una tolerancia de 10^{-8}, ¿cuántos intervalos requiere aproximadamente cada método? ¿Cuál es más eficiente?

Rta: Para alcanzar una tolerancia de 10^{-8}, las cotas teóricas indican que el método del Trapecio requiere aproximadamente 4922 subintervalos, mientras que Simpson necesita 66 subintervalos.

Por lo tanto, teóricamente Simpson es más eficiente. No obstante, para esta integral particular, el Trapecio alcanza experimentalmente la tolerancia con aproximadamente 4 subintervalos y Simpson con 8.

Pregunta 3: Límite Numérico y Precisión de Punto Flotante

Si incrementas n a 10^8, ¿el error disminuirá indefinidamente o comenzará a aumentar? Explica la competencia entre el error de truncamiento y el error de redondeo (IEEE 754).

Rta: Al aumentar el número de subintervalos a 10^8, el error no disminuye indefinidamente. Aunque el error de truncamiento se reduce, los errores de redondeo asociados a la precisión IEEE 754 pueden acumularse.

En consecuencia, el error total puede estabilizarse o aumentar, por lo que incrementar excesivamente el número de subintervalos no garantiza mejores resultados.